

Instructions for completing Sprint 6 - Software Development Tools

Video to Template Example: <https://www.youtube.com/watch?v=bna15Zj6jUI>

Step 1. Set up

1. Create an account on github.com. If you already have an account or created one while going through the [Git and GitHub chapter](#), feel free to skip this step.
2. Create a new git repository with a `README.md` file and a `.gitignore` file (choose a Python template). If you need to recap, please refer to [this lesson](#).
3. If you haven't already, create an account on render.com. Render and web applications were discussed in [this lesson](#). When making a Render account, choose the "GitHub" option and follow the signup steps. This is exactly what you need to link your Render account to your GitHub account.
4. This project involves performing exploratory data analysis. To accomplish this, you will need to have the `pandas` and `plotly-express` packages installed. We previously discussed `plotly` in [this lesson](#). `plotly-express` is designed with reasonable defaults and configurable options for common plot types, making it a great starting point for beginners compared to just `plotly`. If you are new to it, don't worry - we will guide you through the process!

Additionally, you will need the `streamlit` package to develop a web application.

Create a new virtual environment and give it a meaningful name that is related to the dataset you will be working with. For example, you could name it `vehicles_env`.

Ensure that you have installed at least the following packages to the environment: `pandas`, `streamlit`, and `plotly-express`.

- In case you struggle to find them, here are the commands for installing the libraries:
 - For `plotly-express`, check out [this documentation page](#).
 - [Here is the doc page](#) for `pandas` where you will find the command to run.
 - Finally, [this is a page](#) for `streamlit`.

- Besides that, you'll need to install an additional library (`nbformat`) to enable plotly express. Install it via `conda install -n myenv nbformat`
5. If you need a recap on how to create a virtual environment and install packages, refer to [the lesson on this topic](#).
 6. [Install VS Code](#) if you haven't done it yet. Clone your project repository from GitHub and open it as a project in VS Code. This is going to be your project directory. Set the Python interpreter to the one used by the virtual environment you've created previously.
 7. For the sake of simplicity, instead of saving your environment to the `requirements.txt` file to the project directory, manually create the `requirement.txt` file and add three libraries there without specifying versions for them:
 - pandas
 - plotly_express
 - streamlit

Step 2. Download the data file

1. [Download the car advertisement dataset](#) (`vehicles_us.csv`) or find your own dataset in a CSV format.
2. Place the dataset in the project directory.

Step 3. Exploratory Data Analysis

1. Create a directory named `notebooks` in your project directory.
2. Create a Jupyter notebook named `EDA.ipynb` in VS Code and save it to the `notebooks` directory of your project. Remember that `.ipynb` is a file extension for Jupyter Notebooks.
3. Open the `EDA.ipynb` Jupyter notebook and experiment with `plotly-express` to create visualizations for basic exploratory analysis of the dataset within the notebook.

Here are a few examples on:

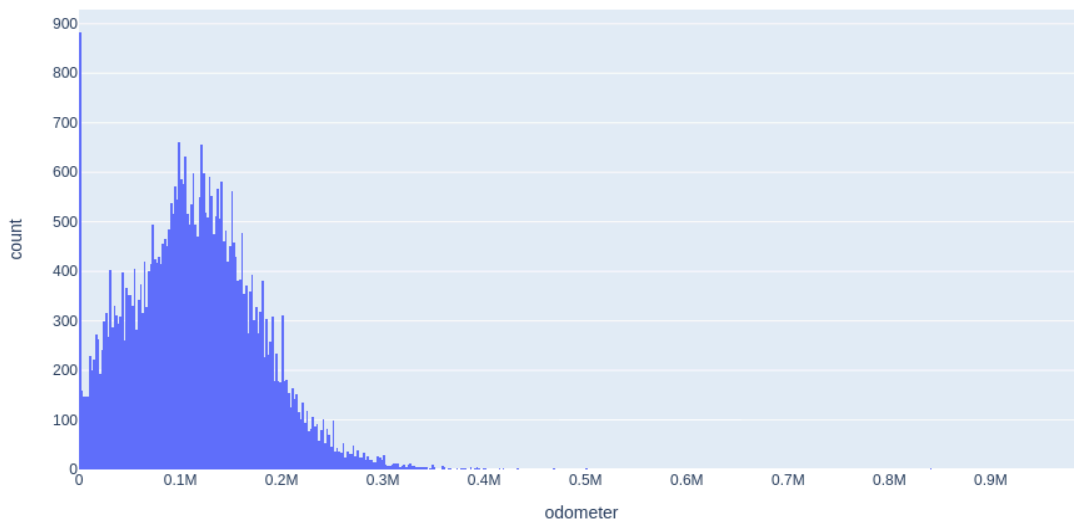
How to build a histogram using *plotly-express*.

```
import pandas as pd
import plotly.express as px

car_data = pd.read_csv('vehicles_us.csv') # reading in the data

fig = px.histogram(car_data, x="odometer") # creating a histogram

fig.show() # displaying it
```

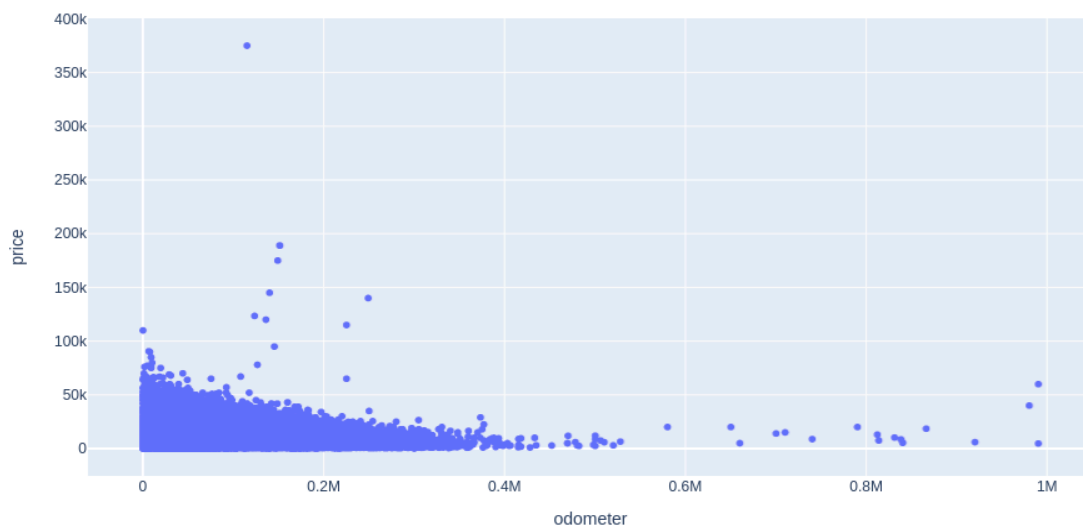


How to build a scatterplot in this library.

```
import pandas as pd
import plotly.express as px

car_data = pd.read_csv('vehicles_us.csv') # reading in the data

fig = px.scatter(car_data, x="odometer", y="price") # creating a scatterplot
fig.show() # displaying it
```



If you're interested in building other types of charts using `plotly-express`, you can find examples of the most common basic plots listed on [this page](#). The page includes examples, as well as the code needed to implement them. However, please don't spend too much time exploring the dataset, as the focus of this project is different. Our goal is to build a web app, and while having many charts and plots in the app would be great, it's not necessary at this stage. Instead, let's focus on the end-to-end work required to deploy such an app. This is exactly what the future steps are about.

Step 4. Develop the web application dashboard

1. Create an `app.py` file in the root directory of your project. To create a `.py` file, click on "New File" in VS Code and save it to the project directory with the desired name and `.py` extension.
2. Import `streamlit` as `st`, `pandas`, and `plotly-express` at the beginning of the file.
3. Read the dataset's CSV file into a DataFrame. The code will be the same as what you had in the Jupyter Notebook when exploring the dataset.
4. Now, let's create the content of our Streamlit-based application. Here is what we want you to include in it:
 - At least one header (you can use `st.header()` to do it. In the [web apps lesson](#), we show how to create a header).
 - Create a button that, when clicked, will build a `plotly-express` histogram. To do this, consider using `st.write()` and `st.plotly_chart()` functions.

Here is an example of how you can do it:

```

import pandas as pd

import plotly.express as px

import streamlit as st

car_data = pd.read_csv('vehicles_us.csv') # reading in the data
hist_button = st.button('Build histogram') # creating a button

if hist_button: # if the button is clicked
    # writing a message
    st.write('Creating a histogram for the car sales advertisements dataset')

    # creating a histogram
    fig = px.histogram(car_data, x="odometer")

    # displaying an interactive Plotly chart
    st.plotly_chart(fig, use_container_width=True)

```

- Add another button that, when clicked, builds a **plotly-express** scatterplot. Again, consider using the [st.write\(\)](#) and [st.plotly_chart\(\)](#) functions

It's optional, but if you're up for an extra challenge, consider replacing buttons with checkboxes, which are available in **streamlit** via **st.checkbox()**. You can prompt the user to select a checkbox corresponding to either a histogram or a scatterplot, and then generate a plot based on the selected checkbox. Here is a simple example of how checkboxes work in **streamlit**:

```

import streamlit as st

# creating a checkbox
build_histogram = st.checkbox('Build a histogram')

if build_histogram: # if the checkbox is selected
    st.write('Building a histogram for the odometer column')
...

```

5. Make sure to update the **README** file when finished. It should include a brief description of the project, explaining what the web application is for and what kind of functionality it provides.
6. To make Streamlit compatible with Render, add a Streamlit configuration file to your project repository at [streamlit/config.toml](#) with the following contents:

[server]

headless = true

port = 10000

[browser]

serverAddress = "0.0.0.0"

serverPort = 10000

It'll tell Render to look in the right place to listen to your streamlit app when hosting it on its servers.

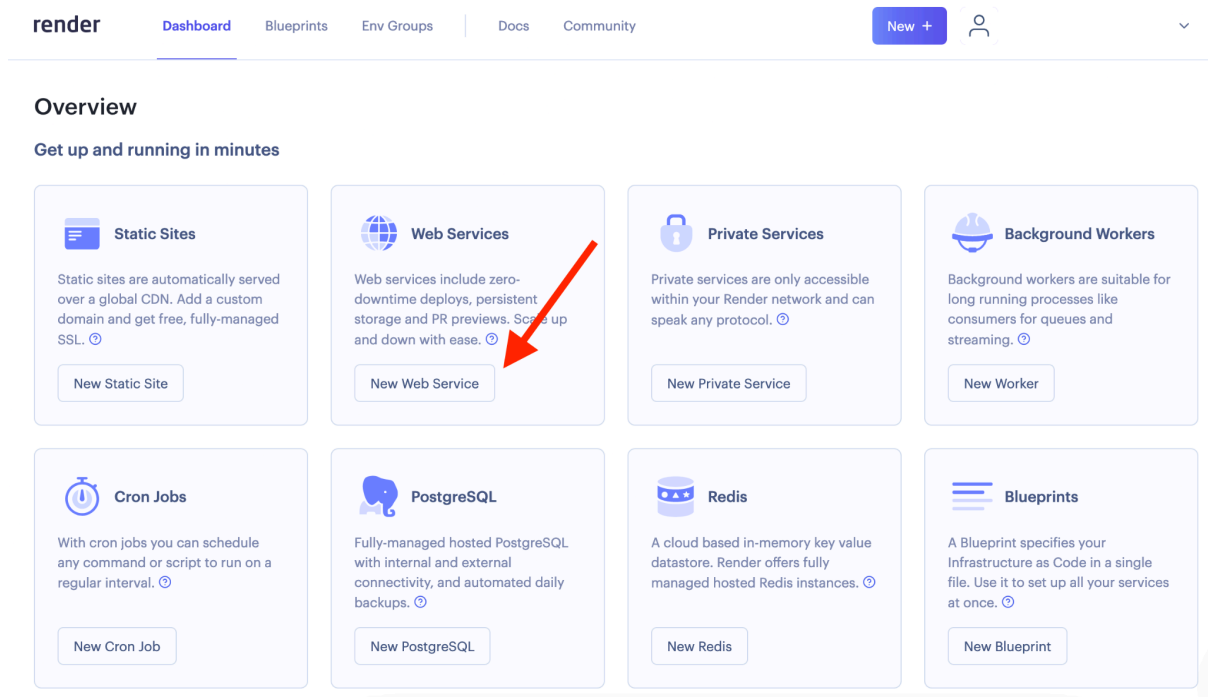
7. Remember to commit and push all changes back to your repository when you are finished with your work. Without doing so, nothing will function properly.

Notes to consider:

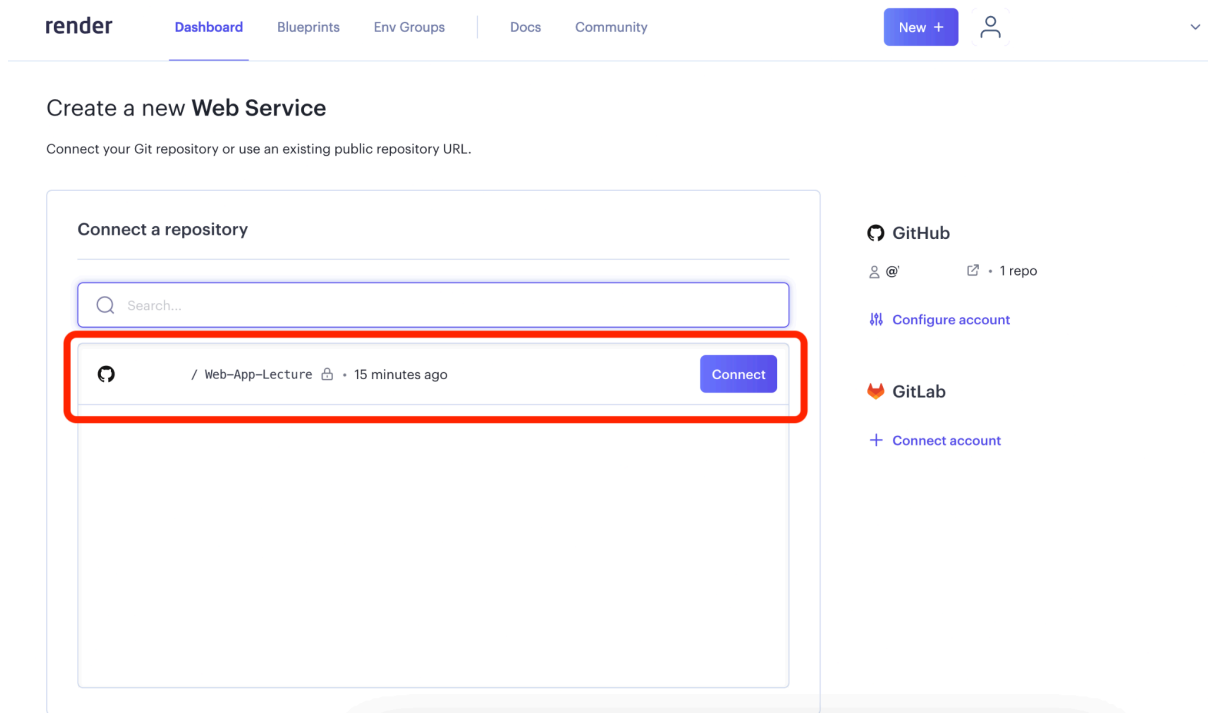
- As you develop your application by adding a new Streamlit component, you can run `streamlit run app.py` command from the terminal to see what the result looks like
- As you reach some milestones in the application development (e.g. you add a working component and the application runs without errors), it's a good practice to commit and push your work to a remote repository on GitHub. So don't forget to write a meaningful commit message!

Step 5. Deploy the final version of the application to Render

1. Open your account on render.com and create a new web service:



2. Create a new web service linked to your Github repository:



3. Configure the new Render web service by adding a **Build Command** that installs everything needed to run your app, including `streamlit` and all packages from `requirements.txt`. Use the following command:

```
pip install --upgrade pip && pip install -r requirements.txt
```

To your **Start Command**, add: `streamlit run app.py`. It should look like this:

The screenshot shows the Render dashboard for a new web service. The service is named 'lecture 8' and is configured for Python 3, Oregon (US West) region, and the main branch. The Build Command is set to `$ pip install streamlit & pip install -r requirements.txt` and the Start Command is set to `$ streamlit run app.py`. Both commands are highlighted with red boxes.

render Dashboard Blueprints Env Groups Docs Community New +

You are deploying a web service for `/Web-App-Lecture.`

You seem to be using Python, so we've autofilled some fields accordingly. Make sure the values look right to you!

Name
A unique name for your web service.
lecture 8

Environment
The runtime environment for your web service.
Python 3

Region
The region where your web service runs.
Oregon (US West)

Branch
The repository branch used for your web service.
main

Build Command
This command runs in the root directory of your repository when a new version of your code is pushed, or when you deploy manually. It is typically a script that installs libraries, runs migrations, or compiles resources needed by your app.
\$ pip install streamlit & pip install -r requirements.txt

Start Command
This command runs in the root directory of your app and is responsible for starting its processes. It is typically used to start a webserver for your app. It can access environment variables defined by you in Render.
\$ streamlit run app.py

4. Deploy to Render, wait for the build to succeed:

The screenshot shows the Render dashboard for the 'lecture 8' web service. The service is in the 'In progress' state. The deployment logs show the build process, including the installation of dependencies and the successful upload of the build. The logs are highlighted with a red box.

render Dashboard Blueprints Env Groups Docs Community New +

WEB SERVICE

lecture 8 Python 3 Free Plan /Web-App-Lecture main Connect Manual Deploy

https://lecture-8.onrender.com

Events Builds too slow? Upgrade to a paid plan to go faster. Learn more about free plan limits.

Logs August 30, 2022 at 1:59 PM In progress

Disks O8da094 streamlit app

Environment

Shell Search logs Go Maximize Scroll to top

PRs

Jobs

Sharing

Metrics

Scaling

Settings

2.28.1 rich-12.5.1 scipy-1.7.3 semver-2.13.0 six-1.16.0 smmap-5.0.0 streamlit-1.12.2 toml-0.10.2 toolz-0.12.0 tornado-6.2 typin
g-extensions-4.3.0 tzdata-2022.2 tzlocal-4.2 urllib3-1.26.12 validators-0.20.0 watchdog-2.1.9 zipp-3.8.1
Aug 30 02:00:52 PM WARNING: You are using pip version 20.1.1; however, version 22.2.2 is available.
Aug 30 02:00:52 PM You should consider upgrading via the '/opt/render/project/src/.venv/bin/python -m pip install --upgrade pi
p' command.
Aug 30 02:00:53 PM ==> Generating container image from build. This may take a few minutes...
Aug 30 02:04:00 PM ==> Uploading build...
Aug 30 02:04:54 PM ==> Build uploaded in 34s
Aug 30 02:04:55 PM ==> Build successful
Aug 30 02:04:55 PM ==> Deploying...

Scroll to bottom

5. Verify that your application is accessible at the following URL:

https://<APP_NAME>.onrender.com/

Note: it can take several minutes after a successful deployment for the app to be available online on a free tier. Also note that apps go “asleep” after being inactive for a few minutes. If so, just load and refresh your app a few times for it to get awoken.

If you update your GitHub repository, to deploy the newest version in Render, click Manual Deploy → Latest Commit

How to submit my project:

You'll need to submit a link to your GitHub repository. Please also add the URL of your app on Render to your project's [README.md](#)

How will my project be evaluated?

We've put together some project assessment criteria. Read over them carefully before you make a submission.

Here's what project reviewers look for when assessing your project:

- Does the project repository contain at least the following files?

```
$ tree
.
├── README.md
├── app.py
├── vehicles_us.csv
├── requirements.txt
├── notebooks
│   └── EDA.ipynb
├── .streamlit
│   └── config.toml
```

- Is the web app accessible via a browser?
- Does the web app contain the following?
 - at least one header with text
 - at least 1 histogram
 - at least 1 scatter plot
 - at least one button or a checkbox